



Chapter 4: MongoDB & Mongoose

Table of Contents

- [4.1 What is MongoDB?](#)
- [4.2 What is Mongoose?](#)
- [4.3 Connection Setup](#)
- [4.4 Schema Definition](#)
- [4.5 Model Creation](#)
- [4.6 CRUD Operations](#)
- [4.7 Query Operators](#)
- [4.8 Population \(Joins\)](#)
- [4.9 Aggregation Pipeline](#)
- [4.10 Middleware \(Hooks\)](#)
- [4.11 Indexes](#)

4.1 What is MongoDB?

Definition: MongoDB is a NoSQL document database that stores data in flexible, JSON-like documents (BSON). Unlike relational databases with fixed schemas and tables, MongoDB allows different structures in the same collection and supports horizontal scaling through sharding.

Key Concepts:

MongoDB	SQL Equivalent
Database	Database
Collection	Table
Document	Row
Field	Column
Embedded Document	Join
_id	Primary Key

```
// Sample MongoDB Document
{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "name": "John Doe",
  "email": "john@example.com",
  "age": 30,
  "address": {
    "city": "New York",
    "country": "USA"
  },
  "hobbies": ["reading", "coding"],
  "createdAt": ISODate("2024-01-15T10:30:00Z")
}
```

4.2 What is Mongoose?

Definition: Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js. It provides schema definition, data validation, type casting, query building, and middleware hooks, making it easier to interact with MongoDB in a structured, type-safe way.

Benefits:

- Schema validation
 - Type casting
 - Query building helpers
 - Middleware/hooks
 - Population (like SQL joins)
 - Plugins support
-

4.3 Connection Setup

```
const mongoose = require('mongoose');

// Connection string
const MONGODB_URI = process.env.MONGODB_URI || 'mongodb://localhost:27017/myapp';

// Connection options
const options = {
  useNewUrlParser: true,
  useUnifiedTopology: true,
  // useCreateIndex: true, // Deprecated in Mongoose 6+
  // useFindAndModify: false // Deprecated in Mongoose 6+
};

// Connect to MongoDB
mongoose.connect(MONGODB_URI, options)
  .then(() => console.log('✅ MongoDB connected'))
  .catch(err => console.error('❌ MongoDB connection error:', err));

// Connection events
mongoose.connection.on('connected', () => {
  console.log('Mongoose connected to DB');
});

mongoose.connection.on('error', (err) => {
  console.error('Mongoose connection error:', err);
});

mongoose.connection.on('disconnected', () => {
  console.log('Mongoose disconnected');
});

// Graceful shutdown
process.on('SIGINT', async () => {
```

```

    await mongoose.connection.close();
    console.log('MongoDB connection closed due to app termination');
    process.exit(0);
  });

// Connection function pattern
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(MONGODB_URI, options);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;

```

4.4 Schema Definition

Definition: A Mongoose schema defines the structure of documents in a collection—field names, types, validators, defaults, and options. It acts as a blueprint that enforces data consistency and validation even though MongoDB itself is schema-less.

```

const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  // ===== BASIC TYPES =====
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    minlength: [2, 'Name must be at least 2 characters'],
    maxlength: [50, 'Name cannot exceed 50 characters']
  },

  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Please use a valid email address']
  },

  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [6, 'Password must be at least 6 characters'],

```

```

    select: false // Won't be returned in queries by default
  },

  age: {
    type: Number,
    min: [0, 'Age cannot be negative'],
    max: [150, 'Age seems too high']
  },

  // ===== ENUM =====
  role: {
    type: String,
    enum: {
      values: ['user', 'admin', 'moderator'],
      message: '{VALUE} is not a valid role'
    },
    default: 'user'
  },

  // ===== BOOLEAN =====
  isActive: {
    type: Boolean,
    default: true
  },

  isVerified: {
    type: Boolean,
    default: false
  },

  // ===== NESTED OBJECT =====
  address: {
    street: String,
    city: { type: String, required: true },
    state: String,
    country: { type: String, default: 'India' },
    zipCode: {
      type: String,
      validate: {
        validator: function(v) {
          return /^\d{5,6}$/.test(v);
        },
        message: props => `${props.value} is not a valid zip code`
      }
    }
  },

  // ===== ARRAY OF STRINGS =====
  hobbies: [String],

  tags: {
    type: [String],

```

```

    validate: {
      validator: function(v) {
        return v.length <= 5;
      },
      message: 'Cannot have more than 5 tags'
    }
  },

  // ===== ARRAY OF OBJECTS =====
  education: [{
    institution: { type: String, required: true },
    degree: String,
    field: String,
    year: {
      type: Number,
      min: 1900,
      max: new Date().getFullYear()
    }
  }],

  // ===== REFERENCE TO ANOTHER MODEL =====
  posts: [{
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Post'
  }],

  createdBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  },

  // ===== DATE =====
  dateOfBirth: Date,

  lastLogin: {
    type: Date,
    default: Date.now
  },

  // ===== MIXED TYPE (Any) =====
  metadata: mongoose.Schema.Types.Mixed,

  // ===== CUSTOM GETTER/SETTER =====
  phone: {
    type: String,
    get: v => v ? `+91-${v}` : v,
    set: v => v.replace(/\D/g, '') // Remove non-digits
  }
}, {
  // ===== SCHEMA OPTIONS =====
  timestamps: true, // Adds createdAt and updatedAt

```

```

    toJSON: {
      virtuals: true,
      transform: function(doc, ret) {
        delete ret.password;
        delete ret.__v;
        return ret;
      }
    },
    toObject: { virtuals: true }
  });

// ===== VIRTUAL PROPERTIES =====
// Not stored in DB, computed on the fly
userSchema.virtual('fullName').get(function() {
  return `${this.firstName} ${this.lastName}`;
});

// Virtual for reverse population
userSchema.virtual('myPosts', {
  ref: 'Post',
  localField: '_id',
  foreignField: 'author'
});

// ===== INSTANCE METHODS =====
userSchema.methods.comparePassword = async function(candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

userSchema.methods.generateAuthToken = function() {
  return jwt.sign({ id: this._id }, process.env.JWT_SECRET, {
    expiresIn: '7d'
  });
};

// ===== STATIC METHODS =====
userSchema.statics.findByEmail = function(email) {
  return this.findOne({ email: email.toLowerCase() });
};

userSchema.statics.getActiveUsers = function() {
  return this.find({ isActive: true });
};

module.exports = userSchema;

```

4.5 Model Creation

Definition: A Model is a compiled version of a schema that provides an interface for CRUD operations on the database. It represents a collection and allows you to create, query, update, and delete documents.

```

const mongoose = require('mongoose');
const userSchema = require('./userSchema');

// Create model from schema
const User = mongoose.model('User', userSchema);
// Collection name will be 'users' (lowercase, pluralized)

// Create model with custom collection name
const Admin = mongoose.model('Admin', adminSchema, 'administrators');

// Export model
module.exports = User;

// Using the model
const user = new User({
  name: 'John Doe',
  email: 'john@example.com',
  password: 'password123'
});

await user.save();

```

4.6 CRUD Operations

Definition: CRUD stands for Create, Read, Update, Delete—the four basic operations for persistent storage. Mongoose provides intuitive methods like `create()`, `find()`, `findByIdAndUpdate()`, and `deleteOne()` that handle these operations with built-in validation.

```

const User = require('./models/User');

// ===== CREATE =====

// Method 1: new + save
const user = new User({
  name: 'John Doe',
  email: 'john@example.com',
  password: 'password123'
});
await user.save();

// Method 2: create (shorthand)
const user = await User.create({
  name: 'Jane Doe',
  email: 'jane@example.com',
  password: 'password456'
});

// Create multiple
const users = await User.insertMany([

```

```

    { name: 'User 1', email: 'user1@example.com' },
    { name: 'User 2', email: 'user2@example.com' }
  ]);

// ===== READ =====

// Find all
const allUsers = await User.find();

// Find with conditions
const activeUsers = await User.find({ isActive: true });

// Find with multiple conditions
const filteredUsers = await User.find({
  isActive: true,
  role: 'admin',
  age: { $gte: 18 }
});

// Find one (first match)
const user = await User.findOne({ email: 'john@example.com' });

// Find by ID
const user = await User.findById('507f1f77bcf86cd799439011');
const user = await User.findById(req.params.id);

// Select specific fields
const users = await User.find().select('name email -_id');
const users = await User.find().select('-password -__v');

// Sorting
const users = await User.find().sort({ createdAt: -1 }); // Descending
const users = await User.find().sort('name'); // Ascending
const users = await User.find().sort('-createdAt name'); // Multiple

// Pagination
const page = parseInt(req.query.page) || 1;
const limit = parseInt(req.query.limit) || 10;
const skip = (page - 1) * limit;

const users = await User.find()
  .sort('-createdAt')
  .skip(skip)
  .limit(limit);

const total = await User.countDocuments();
const totalPages = Math.ceil(total / limit);

// Lean (returns plain JS objects, faster)
const users = await User.find().lean();

// ===== UPDATE =====

```



```

// findByIdAndUpdate
const updatedUser = await User.findByIdAndUpdate(
  id,
  { name: 'New Name', age: 31 },
  {
    new: true,           // Return updated document
    runValidators: true  // Run schema validators
  }
);

// findOneAndUpdate
const updatedUser = await User.findOneAndUpdate(
  { email: 'john@example.com' },
  { $set: { isActive: false } },
  { new: true }
);

// updateOne (doesn't return document)
const result = await User.updateOne(
  { _id: id },
  { $set: { name: 'Updated' } }
);
// result: { acknowledged: true, modifiedCount: 1, ... }

// updateMany
const result = await User.updateMany(
  { isActive: false },
  { $set: { role: 'inactive' } }
);

// Using save (triggers middleware)
const user = await User.findById(id);
user.name = 'New Name';
user.age = 31;
await user.save();

// ===== DELETE =====

// findByIdAndDelete
const deletedUser = await User.findByIdAndDelete(id);

// findOneAndDelete
const deletedUser = await User.findOneAndDelete({ email: 'john@example.com' });

// deleteOne
const result = await User.deleteOne({ _id: id });
// result: { acknowledged: true, deletedCount: 1 }

// deleteMany
const result = await User.deleteMany({ isActive: false });

```

```
// ===== OTHER USEFUL METHODS =====

// Count
const count = await User.countDocuments({ isActive: true });

// Check if exists
const exists = await User.exists({ email: 'john@example.com' });

// Distinct values
const roles = await User.distinct('role');
// ['user', 'admin', 'moderator']

// Find or Create
let user = await User.findOne({ email });
if (!user) {
  user = await User.create({ email, name });
}
```

4.7 Query Operators

Definition: MongoDB query operators are special keywords prefixed with `$` that perform comparisons, logical operations, array queries, or element checks within queries. They enable complex querying beyond simple equality matches.

```
// ===== COMPARISON OPERATORS =====

// Equal (implicit)
User.find({ age: 25 });

// Equal (explicit)
User.find({ age: { $eq: 25 } });

// Not equal
User.find({ age: { $ne: 25 } });

// Greater than
User.find({ age: { $gt: 25 } });

// Greater than or equal
User.find({ age: { $gte: 25 } });

// Less than
User.find({ age: { $lt: 25 } });

// Less than or equal
User.find({ age: { $lte: 25 } });

// In array of values
User.find({ role: { $in: ['admin', 'moderator'] } });
```

```

// Not in array
User.find({ role: { $nin: ['banned', 'suspended'] } });

// Range
User.find({ age: { $gte: 18, $lte: 65 } });

// ===== LOGICAL OPERATORS =====

// AND (implicit - multiple conditions)
User.find({ age: { $gte: 18 }, isActive: true });

// AND (explicit)
User.find({
  $and: [
    { age: { $gte: 18 } },
    { age: { $lte: 65 } },
    { isActive: true }
  ]
});

// OR
User.find({
  $or: [
    { role: 'admin' },
    { role: 'moderator' }
  ]
});

// NOT
User.find({
  age: { $not: { $gt: 25 } }
});

// NOR (neither)
User.find({
  $nor: [
    { age: 25 },
    { name: 'John' }
  ]
});

// ===== ELEMENT OPERATORS =====

// Field exists
User.find({ phone: { $exists: true } });
User.find({ phone: { $exists: false } });

// Type check
User.find({ age: { $type: 'number' } });
User.find({ age: { $type: 16 } }); // 16 = int32

// ===== ARRAY OPERATORS =====

```

```

// Array contains value
User.find({ hobbies: 'reading' });

// Array contains all values
User.find({ hobbies: { $all: ['reading', 'coding'] } });

// Array size
User.find({ hobbies: { $size: 3 } });

// Array element match
User.find({
  education: {
    $elemMatch: {
      institution: 'MIT',
      year: { $gte: 2020 }
    }
  }
});

// ===== STRING/REGEX =====

// Regex pattern
User.find({ name: /^john/i }); // Starts with john (case-insensitive)
User.find({ email: { $regex: '@gmail.com$' } }); // Ends with

// ===== TEXT SEARCH =====
// Requires text index: userSchema.index({ name: 'text', bio: 'text' })
User.find({ $text: { $search: 'developer javascript' } });

// ===== UPDATE OPERATORS =====

// $set - Set field value
User.updateOne({ _id: id }, { $set: { name: 'New Name' } });

// $unset - Remove field
User.updateOne({ _id: id }, { $unset: { tempField: 1 } });

// $inc - Increment
User.updateOne({ _id: id }, { $inc: { loginCount: 1 } });
User.updateOne({ _id: id }, { $inc: { balance: -50 } }); // Decrement

// $mul - Multiply
User.updateOne({ _id: id }, { $mul: { price: 1.1 } }); // 10% increase

// $min/$max - Update if new value is less/greater
User.updateOne({ _id: id }, { $min: { lowScore: 50 } });
User.updateOne({ _id: id }, { $max: { highScore: 100 } });

// $rename - Rename field
User.updateOne({ _id: id }, { $rename: { 'name': 'fullName' } });

```

```

// $push - Add to array
User.updateOne({ _id: id }, { $push: { hobbies: 'gaming' } });

// $push with modifiers
User.updateOne({ _id: id }, {
  $push: {
    scores: {
      $each: [90, 95, 85],
      $sort: -1,
      $slice: 5 // Keep only top 5
    }
  }
});

// $addToSet - Add if not exists
User.updateOne({ _id: id }, { $addToSet: { hobbies: 'reading' } });

// $pull - Remove from array
User.updateOne({ _id: id }, { $pull: { hobbies: 'reading' } });

// $pop - Remove first/last element
User.updateOne({ _id: id }, { $pop: { hobbies: 1 } }); // Last
User.updateOne({ _id: id }, { $pop: { hobbies: -1 } }); // First

// $pullAll - Remove all matching
User.updateOne({ _id: id }, { $pullAll: { hobbies: ['a', 'b'] } });

```

4.8 Population (Joins)

Definition: Population is Mongoose's way of referencing documents in other collections and automatically replacing the reference IDs with the actual document data when querying. It's similar to SQL joins but handled at the application level.

```

// ===== SCHEMA SETUP =====

// User Schema
const userSchema = new mongoose.Schema({
  name: String,
  email: String
});

// Post Schema with reference
const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },

```

```

    comments: [{
      type: mongoose.Schema.Types.ObjectId,
      ref: 'Comment'
    }]
  });

// Comment Schema
const commentSchema = new mongoose.Schema({
  text: String,
  author: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User'
  },
  post: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Post'
  }
});

// ===== BASIC POPULATION =====

// Populate single reference
const post = await Post.findById(postId).populate('author');
// post.author is now { _id: ..., name: 'John', email: '...' }

// Populate with selected fields
const post = await Post.findById(postId)
  .populate('author', 'name email'); // Only name and email

// Exclude fields
const post = await Post.findById(postId)
  .populate('author', '-password -__v');

// ===== MULTIPLE POPULATION =====

const post = await Post.findById(postId)
  .populate('author')
  .populate('comments');

// Or in one call
const post = await Post.findById(postId)
  .populate(['author', 'comments']);

// ===== NESTED POPULATION =====

const post = await Post.findById(postId)
  .populate({
    path: 'comments',
    populate: {
      path: 'author',
      select: 'name'
    }
  })

```

```

    });

// ===== POPULATION WITH CONDITIONS =====

const post = await Post.findById(postId)
    .populate({
        path: 'comments',
        match: { approved: true }, // Only approved comments
        select: 'text author',
        options: {
            sort: { createdAt: -1 },
            limit: 5
        },
        populate: { path: 'author', select: 'name' }
    });

// ===== VIRTUAL POPULATION =====
// Define virtual in userSchema
userSchema.virtual('posts', {
    ref: 'Post',
    localField: '_id',
    foreignField: 'author'
});

// Enable virtuals in toJSON
userSchema.set('toJSON', { virtuals: true });

// Query with virtual population
const user = await User.findById(userId)
    .populate('posts');
// user.posts is now array of posts by this user

```

4.9 Aggregation Pipeline

Definition: Aggregation pipeline is MongoDB's powerful framework for data analysis and transformation. It processes documents through multiple stages (match, group, sort, project, etc.) where each stage transforms the documents as they pass through.

```

// ===== BASIC AGGREGATION =====

const stats = await User.aggregate([
    // Stage 1: Match (filter documents)
    {
        $match: { isActive: true }
    },

    // Stage 2: Group (aggregate data)
    {
        $group: {
            _id: '$role',

```

```

        count: { $sum: 1 },
        avgAge: { $avg: '$age' },
        minAge: { $min: '$age' },
        maxAge: { $max: '$age' },
        totalSalary: { $sum: '$salary' },
        users: { $push: '$name' }
    }
},

// Stage 3: Sort
{ $sort: { count: -1 } },

// Stage 4: Project (reshape output)
{
    $project: {
        role: '$_id',
        _id: 0,
        count: 1,
        avgAge: { $round: ['$avgAge', 1] }
    }
}
]);

// ===== LOOKUP (Join) =====

const usersWithPosts = await User.aggregate([
    {
        $lookup: {
            from: 'posts',           // Collection name (not model name)
            localField: '_id',
            foreignField: 'author',
            as: 'posts'
        }
    },
    {
        $addFields: {
            postCount: { $size: '$posts' }
        }
    }
]);

// ===== UNWIND =====
// Deconstructs an array, creating one document per element

const hobbyStats = await User.aggregate([
    { $unwind: '$hobbies' },
    {
        $group: {
            _id: '$hobbies',
            count: { $sum: 1 }
        }
    }
],

```



```

    { $sort: { count: -1 } }
  ]]);
// [{ _id: 'reading', count: 15 }, { _id: 'coding', count: 12 }, ...]

// ===== DATE AGGREGATION =====

const monthlyRegistrations = await User.aggregate([
  {
    $group: {
      _id: {
        year: { $year: '$createdAt' },
        month: { $month: '$createdAt' }
      },
      count: { $sum: 1 }
    }
  },
  { $sort: { '_id.year': -1, '_id.month': -1 } },
  { $limit: 12 }
]);

// ===== FACET (Multiple Pipelines) =====

const result = await Product.aggregate([
  {
    $facet: {
      // Pipeline 1: Category stats
      byCategory: [
        { $group: { _id: '$category', count: { $sum: 1 } } }
      ],
      // Pipeline 2: Price ranges
      byPrice: [
        {
          $bucket: {
            groupBy: '$price',
            boundaries: [0, 100, 500, 1000, Infinity],
            default: 'Other',
            output: { count: { $sum: 1 } }
          }
        }
      ],
      // Pipeline 3: Total count
      total: [
        { $count: 'count' }
      ]
    }
  }
]);

// ===== TEXT SEARCH SCORE =====

const results = await Post.aggregate([
  {

```

```

    $match: { $text: { $search: 'javascript tutorial' } }
  },
  {
    $addFields: { score: { $meta: 'textScore' } }
  },
  { $sort: { score: -1 } },
  { $limit: 10 }
]);

```

4.10 Middleware (Hooks)

Definition: Mongoose middleware are functions that run before or after certain operations like `save`, `validate`, `remove`, or query operations. They're useful for password hashing, logging, cleanup, or any pre/post-processing of documents.

```

const bcrypt = require('bcryptjs');

// ===== DOCUMENT MIDDLEWARE =====
// Runs on: save, validate, remove, init

// Pre-save: Hash password
userSchema.pre('save', async function(next) {
  // Only hash if password is modified
  if (!this.isModified('password')) return next();

  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// Pre-save: Set slug
postSchema.pre('save', function(next) {
  if (this.isModified('title')) {
    this.slug = this.title
      .toLowerCase()
      .replace(/^[a-z0-9]+/g, '-')
      .replace(/(^-|-$/g, '');
  }
  next();
});

// Post-save: Log
userSchema.post('save', function(doc, next) {
  console.log(`User ${doc.name} was saved`);
  next();
});

// Pre-remove: Cleanup related documents
userSchema.pre('remove', async function(next) {
  await Post.deleteMany({ author: this._id });
  await Comment.deleteMany({ author: this._id });
});

```

```

    next();
  });

// ===== QUERY MIDDLEWARE =====
// Runs on: find, findOne, findOneAndUpdate, etc.

// Exclude inactive users from all finds
userSchema.pre(/^find/, function(next) {
  // 'this' is the query object
  this.find({ isActive: { $ne: false } });
  next();
});

// Populate author on find
postSchema.pre(/^find/, function(next) {
  this.populate('author', 'name email');
  next();
});

// Query timing
userSchema.pre('find', function(next) {
  this._startTime = Date.now();
  next();
});

userSchema.post('find', function(docs, next) {
  console.log(`Query took ${Date.now() - this._startTime}ms`);
  next();
});

// ===== AGGREGATE MIDDLEWARE =====

userSchema.pre('aggregate', function(next) {
  // Add match stage at beginning
  this.pipeline().unshift({ $match: { isActive: true } });
  next();
});

// ===== ERROR HANDLING MIDDLEWARE =====

userSchema.post('save', function(error, doc, next) {
  if (error.name === 'MongoServerError' && error.code === 11000) {
    next(new Error('Email already exists'));
  } else {
    next(error);
  }
});

```

4.11 Indexes

Definition: Indexes are data structures that store a subset of the collection's data in an easy-to-traverse form. They dramatically improve query performance by allowing MongoDB to find documents without scanning every document in a collection.

```
// ===== SINGLE FIELD INDEX =====
userSchema.index({ email: 1 }); // Ascending
userSchema.index({ createdAt: -1 }); // Descending

// ===== COMPOUND INDEX =====
userSchema.index({ lastName: 1, firstName: 1 });

// ===== UNIQUE INDEX =====
userSchema.index({ email: 1 }, { unique: true });

// ===== TEXT INDEX =====
postSchema.index({ title: 'text', content: 'text' });

// Search using text index
Post.find({ $text: { $search: 'javascript' } });

// ===== SPARSE INDEX =====
// Only index documents that have the field
userSchema.index({ phone: 1 }, { sparse: true });

// ===== TTL INDEX =====
// Auto-delete documents after specified time
sessionSchema.index({ createdAt: 1 }, { expireAfterSeconds: 3600 });

// ===== GEOSPATIAL INDEX =====
storeSchema.index({ location: '2dsphere' });

// Query nearby
Store.find({
  location: {
    $near: {
      $geometry: { type: 'Point', coordinates: [lng, lat] },
      $maxDistance: 10000 // 10km
    }
  }
});

// ===== EXPLAIN QUERY =====
const explanation = await User.find({ email: 'test@example.com' })
  .explain('executionStats');

console.log(explanation.executionStats.executionTimeMillis);
```

Next: [Chapter 5 - React.js](#)